

Gen AI 활용 웹앱 개발

학습 체크리스트 (1/2)

☐ 생성형 AI 활용의 세 가지 맥락 구분

☐ Provider, API Key, Token의 역할 이해

☐ 무료 API Rate Limit와 대표 에러 코드 이해

☐ 프론트엔드 API Key 노출 위험 이해

학습 체크리스트 (2/2)

☐ 서버 Wrapping 구조 이해

☐ Express 서버의 API 중계 역할 이해

☐ Render 기반 Node 서버 배포 흐름 이해

☐ CORS, SOP, Origin 관계 구분

생성형 AI 활용의 세 가지 맥락

- 프로젝트 운영과 학습 보조
- 에이전트 도구 기반 코드 작업
- API 기반 앱 기능 통합
- 오늘의 초점: API 기반 기능 통합

프로젝트 운영과 학습 보조

- 프로젝트 아이디어 정리
- 일정 분해와 학습 로드맵 작성
- README, API 문서, 발표 자료, 에셋 제작
- 결과물 성격: 개발 과정의 보조 산출물

에이전트 도구 기반 코드 작업

- 코드 작성 보조
- 리팩터링 방향 제안
- 리뷰와 버그 위험 탐지
- 테스트 케이스 초안 작성

앱 기능 통합

- 사용자 입력 기반 Provider API 호출
- 대표 기능: 챗봇, 검색, 설명 생성, 요약
- 사용량 기준: 실제 사용자 행동
- 설계 대상: 비용, 한도, API Key 보안

Provider가 필요한 이유

프로바이더 (Provider)

AI 모델을 직접 운영하지 않고 API 형태로 사용할 수 있게 제공하는 서비스 사업자

- 로컬 모델 서빙의 높은 운영 난이도
- GPU, 메모리, 배포 운영 지식 필요
- 웹앱 실습에 적합한 Provider API 방식
- 대표 Provider: OpenAI, Anthropic, Google, Groq

API Key의 역할

API Key (API Key)

API 호출 권한과 사용량 식별을 위한 인증 문자열

- 비밀번호처럼 관리해야 하는 민감정보
- Provider 계정 식별 기준
- 사용량 초과, 오남용, 과금 연결 기준

Token의 의미

토큰 (Token)

AI 모델이 입력과 출력을 처리하기 위해 문장을 작게 나눈 텍스트 단위

- 비용 기준: 요청 횟수와 Token 수
- Token 증가 요인: 긴 프롬프트, 긴 답변
- 최적화 방향: 입력 길이 축소
- 기대 효과: 비용 절감, 응답 시간 단축

무료 API의 적합한 쓰임

- 적합한 목적: 포트폴리오, 기술 검증, 수업 실습
- 운영 한계: 한도, 속도, 모델 변경 위험
- 변동 요소: 계정, 프로젝트, 모델, 시점
- 실습 전 확인: Provider 콘솔의 현재 한도

Rate Limit 읽기

Rate Limit (Rate Limit)

정해진 시간 안에 허용되는 API 호출 횟수 또는 사용량

- RPM: 1분당 요청 수
- RPD: 하루 요청 수
- 한도 초과 결과: `429 Too Many Requests`
- 링크: [Gemini API Rate Limits](#)

대표 HTTP 에러 코드

- 400 Bad Request : 잘못된 요청 형식 또는 입력값
- 401 Unauthorized : API Key 누락 또는 인증 실패
- 403 Forbidden : 인증 후 권한 부족
- 429 Too Many Requests : Rate Limit 또는 사용량 한도 초과
- 500, 503 : Provider 서버 내부 오류 또는 일시적 사용 불가

Google AI Studio

- 역할: Gemini API 실험과 API Key 발급 진입점
- 활용 순서: 프롬프트 검증 후 웹앱 연결
- API Key 발급 위치: AI Studio API Keys 화면
- 모델별 한도 확인 위치: Rate Limit 화면

Google AI Studio 링크

- 새 채팅
- 링크: [Google AI Studio 새 채팅](#)
- API Key 발급
- 링크: [Google AI Studio API Keys](#)
- Rate Limit 확인
- 링크: [Google AI Studio Rate Limit](#)

Gemini 모델 계열

- 성격: Google의 주력 폐쇄형 모델 계열
- 라인업: Pro, Flash, Flash-Lite (Flash Lite 3.1 외엔 테스트 어려움)
- 강점: 멀티모달, 긴 컨텍스트, 범용 웹앱 기능
- 주의: 호출 정도의 테스트 가능한 한도
- 링크: [Gemini API 문서](#)

Gemma 모델 계열

- 성격: Google의 오픈 모델 계열
- 적합 목적: 로컬 실행, 튜닝, 실험, 경량 모델 비교
- 주의: API 제공 여부와 한도 변동 가능성
- 실습 전 확인: 지원 모델과 Rate Limit
- 링크: [AI Studio Rate Limit](#)

Groq

- 성격: 빠른 추론 중심 AI API Provider
- 강점: OSS / Open-weight 모델 활용
- 실습 장점: 모델 교체와 응답 속도 비교
- 확인 대상: 모델별 가격과 Rate Limit

Groq 링크

- 공식 사이트
- 링크: [Groq](#)
- 가격 정보
- 링크: [Groq Pricing](#)
- 사용량 한도
- 링크: [Groq Rate Limits](#)

대표적 Groq 제공 모델 (이후 변동 가능)

- `gpt-oss-20b` : 빠른 응답, 간단한 생성·요약
- `gpt-oss-120b` : 복잡한 추론, 긴 답변 생성
- `qwen/qwen3-32b` : 다국어, 코딩, 문서 생성
- `llama-4-scout-17b-16e-instruct` : 챗봇, 요약, 질의응답

프론트엔드 Key 노출 위험

- 공개 대상: 브라우저로 내려간 JavaScript 파일
- 확인 가능 위치: DevTools Network 탭
- 장기 노출 위치: GitHub commit history
- 결과: 사용량 소진, 과금, 계정 제한

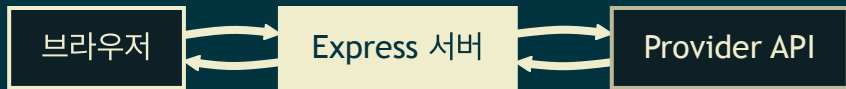
서버 Wrapping

Wrapping (Wrapping)

외부 Provider API를 우리 서버 API로 한 번 감싸서 호출하는 구조

- 클라이언트 호출 대상: `/api/chat`
- 서버 역할: `.env` 의 API Key로 Provider API 대리 호출
- 클라이언트 전달 값: Provider 응답 결과
- 핵심 효과: 브라우저 API Key 노출 방지

Wrapping 흐름



Wrapping 구현 선택지

- 서버리스 함수: Supabase Edge Functions, AWS Lambda, Cloudflare Workers
- 경량 서버: Render, Koyeb, Railway
- 클라우드 인스턴스: GCP, Oracle 등
- 이번 실습 초점: Express 서버와 Render 배포

Express의 역할

Express (Express)

Node.js에서 HTTP 요청과 응답을 쉽게 처리하도록 돕는 경량 서버 프레임워크

- 요청 처리: GET, POST 라우트
- Provider 중계: API Key 포함 요청 재전달
- 활용 형태: 클라이언트와 외부 API 사이의 Proxy 서버

서버 패키지 구성

- `express` : API 라우트 구성
- `cors` : 허용 Origin 제어
- `dotenv` : `.env` 민감정보 로드
- `nodemon` : 개발 중 서버 자동 재시작

SDK를 붙이는 위치

- 실행 위치: 브라우저가 아닌 서버 코드
- 호출 지점: Express 라우트 내부
- SDK 장점: API 주소, 인증 Header, 응답 파싱 단순화
- 링크: [@google/genai](#)
- 링크: [groq-sdk](#)

Postman으로 먼저 테스트

- 목적: 프론트엔드 연결 전 서버 API 단독 검증
- 요청 대상: `POST /api/chat`
- 요청 Body: 테스트 메시지
- 확인 항목: 인증, Rate Limit, Provider 응답 오류
- 링크: [Postman](#)

Origin

Origin (Origin)

브라우저가 출처를 구분할 때 사용하는 프로토콜, 도메인, 포트의 조합

- 비교 대상 1: `http://localhost:5173`
- 비교 대상 2: `http://localhost:3000`
- 차이 기준: 포트가 다른 서로 다른 Origin
- 배포 상황: GitHub Pages와 Render API 서버

SOP

동일 출처 정책 (Same-Origin Policy)

브라우저가 다른 Origin의 리소스 접근을 기본적으로 제한하는 보안 정책

- 목적: 악성 페이지의 외부 데이터 접근 차단
- 주요 적용 위치: 브라우저 기반 요청
- 문제 상황: 프론트엔드와 API 서버의 Origin 불일치

CORS

교차 출처 리소스 공유 (CORS)

서버가 허용한 Origin에 한해 브라우저의 교차 출처 요청을 허용하는 정책

- 역할: 브라우저 제한에 대한 서버 측 허용 선언
- 위험 설정: 모든 Origin 허용
- 운영 방식: 허용 Origin Whitelist 관리

Render 배포 흐름

- 지원 환경: Node, Python, Docker 기반 Web Service
- 연결 방식: GitHub 저장소 연결
- 설정 항목: Build Command, Start Command
- API Key 위치: Render Environment Variables
- 링크: [Render Docs](#)

Render Free Plan 주의점

- 적합 목적: 실습, 포트폴리오 검증
- 부적합 목적: 안정적 프로덕션 운영
- Spin Down: 15분 무트래픽 후 인스턴스 중지
- 확인 위치: Dashboard의 사용량 화면
- 링크: [Render Free Plan](#)

Region 선택

- 서울 리전: 미제공
- 우선 검토: Singapore 또는 Oregon
- 변경 사유: Provider API 리전별 동작 차이
- 확인 대상: 사용 가능한 Render 리전 목록
- 링크: [Render Regions](#)